



SÃO
PAULO
TECH
SCHOOL

Qualidade de Software

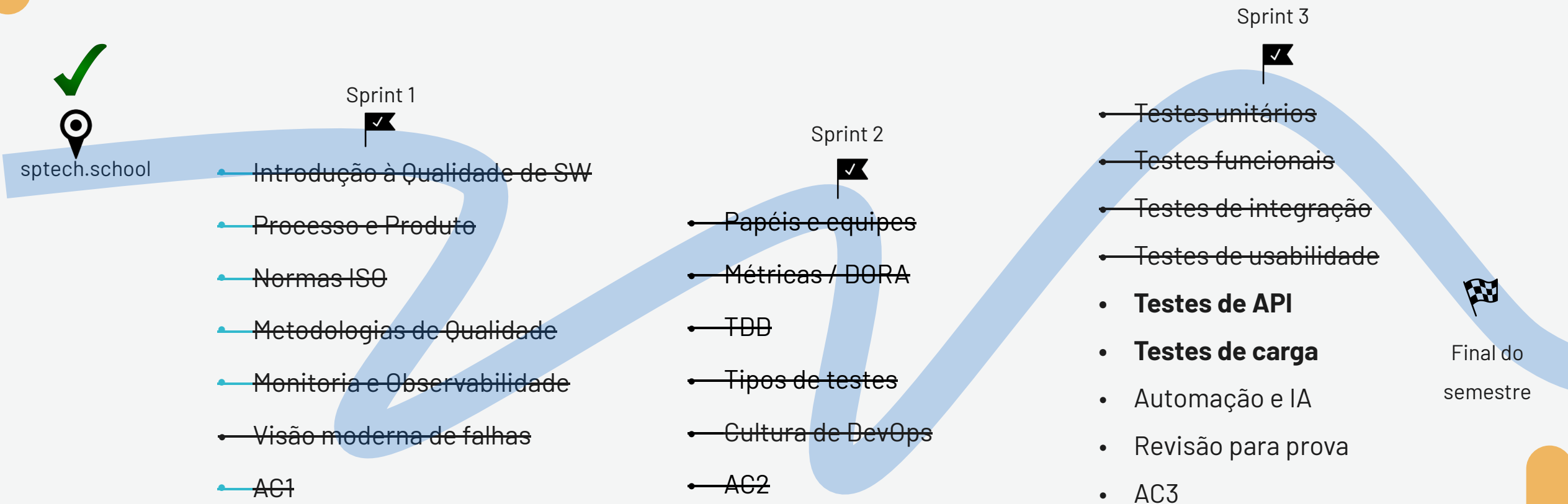
Testes de carga

Testes de API

19/mai

Prates

Nosso caminho - Ementa



Tópicos da Aula

O que são Testes de carga?

Por que fazer testes de carga?

Pontos de atenção

Ferramentas e métricas de sucesso

Principais problemas

O que são Testes de API?

Ferramentas

Atividade

Testes de carga

The background is a dark, monochromatic abstract composition. It features a series of thin, white, wavy lines that flow across the frame, creating a sense of movement and depth. Interspersed among these lines are numerous small, out-of-focus light points, resembling bokeh or distant stars, which add texture and visual interest to the overall design.



O que são Testes de carga?

- Tipo de teste de performance
- Simula múltiplos usuários simultâneos
- Mede comportamento do sistema sob pressão
- Avalia escalabilidade e tempo de resposta

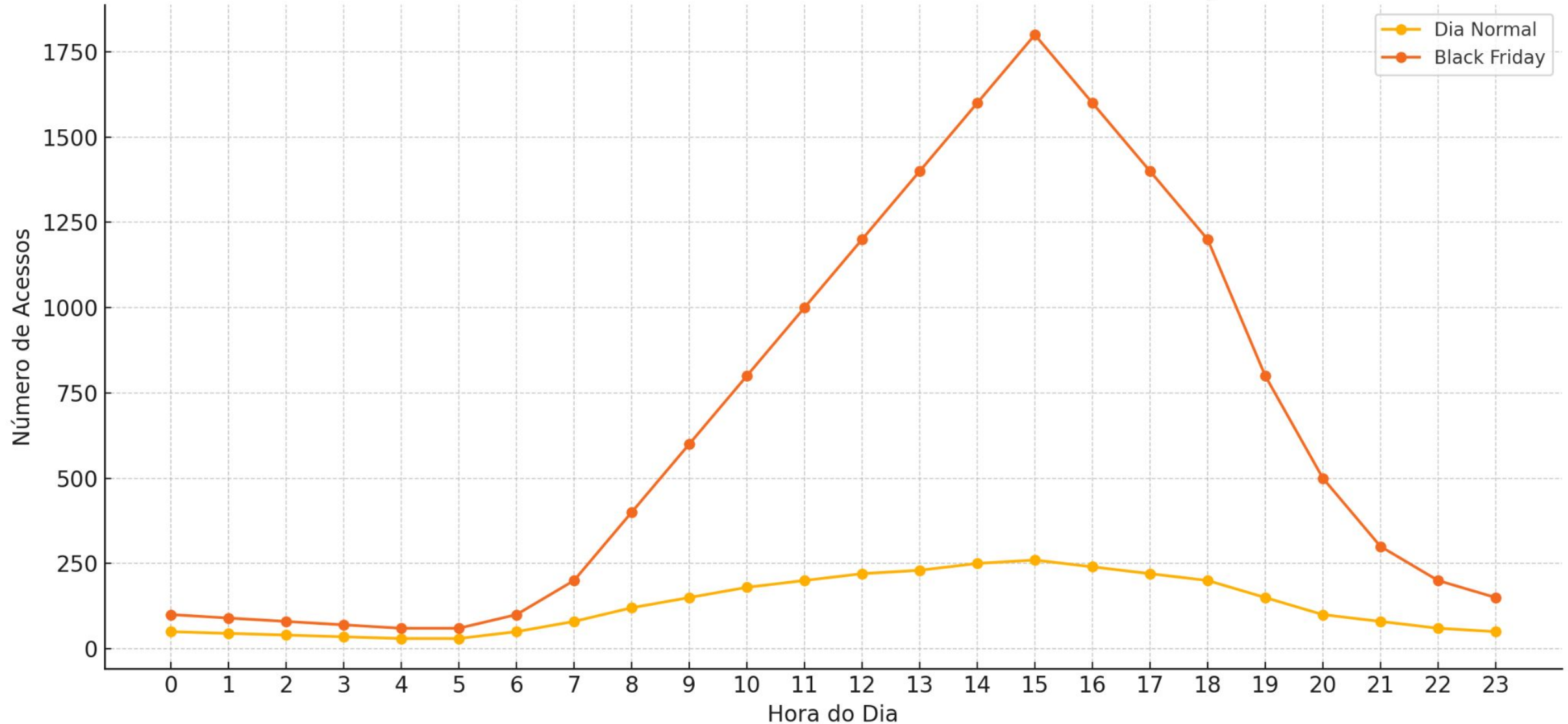


Importância dos testes de carga

- Garantir a estabilidade do sistema em produção
- Prevenir falhas em momentos críticos (ex: Black Friday)
- Identificar gargalos antes que impactem o usuário final
- Validar estratégias de escalonamento de infraestrutura
- Validar custos de ambientes auto escaláveis
- Previsibilidade para gestão

Diferença de acessos - operação regular vs. BF

Comparativo de Acessos: Dia Normal vs Black Friday





Pontos de atenção no planejamento

- Definir a carga-alvo (quantidade de usuários)
- Considerar dependências externas (ex: APIs, banco de dados)
- Evitar short-circuit (falhas que impedem a execução completa do teste)
- Gerenciar logs e métricas (limpeza e monitoramento)
- Considerar autoescalamento e limites de infraestrutura
- Avisar com antecedência



Ferramentas

- Java: Apache JMeter, Gatling
- Python: Locust, PyTest + Locust plugin
- C#: Visual Studio Load Test (deprecated), k6 via CLI
- JavaScript/Node.js: Artillery, k6 (com JS)



Métricas de sucesso

- Tempo médio de resposta aceitável
- Erros de requisição abaixo de um limite (ex: <1%)
- Throughput (requisições/segundo) consistente
- Sem timeouts ou falhas críticas
- Sistema mantém estabilidade sob carga



Estratégia de execução

- Rodar em horários alternativos (fora do expediente, porém considerar outros processos que rodem)
- Usar ambientes semelhantes ao de produção
- Simular cenários reais (horários de pico, eventos)
- Executar progressivamente (carga crescente)
- Automatizar execução via pipeline (CI/CD)



Principais problemas

- Respostas lentas ou timeouts
- Quedas de serviços por falta de escalabilidade
- Erros 500 ou falhas em componentes
- Vazamentos de memória
- Falhas em serviços externos sem fallback

Testes de API

The background is a dark, monochromatic abstract composition. It features a series of thin, white, wavy lines that flow across the frame, creating a sense of movement and depth. Interspersed among these lines are numerous small, out-of-focus white circles, resembling bokeh or distant stars. The overall effect is a sophisticated and modern aesthetic.



O que são Testes de API?

- Testes de API validam a comunicação entre sistemas
- Verificam se os endpoints respondem corretamente
- Avaliam métodos HTTP (GET, POST, PUT, DELETE...)
- São rápidos, automatizáveis e independentes da interface
- Podem ser feitos antes da interface estar pronta
(desenvolvimento guiado por contrato)



Pontos de atenção

Cuidados

- Verificar contratos da API (Swagger/OpenAPI)
- Usar dados de teste válidos e inválidos
- Testar cenários positivos e negativos (ex: 404, 401, 500)
- Considerar dependências externas (serviços, banco)

Estratégias de testes

- Criar casos de uso reais: cliente, produto, pagamento, etc.
- Automatizar suites de testes e incluir no CI/CD
- Utilizar mocks/stubs para isolar dependências
- Priorizar testes críticos (ex: autenticação, criação de dados)



Ferramentas

- Postman – testes manuais e automatizados com collection runner
- Mockito – simulação de respostas/mocks em testes unitários (Java)
- Rest Assured, SuperTest, Pytest + requests

Atividade

The background is a dark, textured surface with intricate, flowing, and swirling patterns of light gray lines. These lines create a sense of movement and depth, resembling a topographical map or a fluid simulation. Scattered throughout the composition are numerous small, bright white dots and larger, soft, out-of-focus circular bokeh lights in shades of gray, adding to the ethereal and dynamic feel of the image.



Exercício

- Simular 500 usuários simultâneos acessando e preenchendo um formulário simples de inscrição
- Pré-requisitos
 - Python 3.7+
 - Locust
 - Flask
- Tarefas: fazer o formulário de inscrição, configurar o teste (ferramenta, qtd de usuarios, etc), executar o teste, avaliar os resultados, gerar um relatório com os resultados

Exemplo - formulário de inscrições

```
from flask import Flask, request, render_template_string

app = Flask(__name__)

# Formulário HTML simples
form_html = """
<!doctype html>
<title>Formulário de Teste</title>
<h1>Inscrição</h1>
<form action="/submit" method="post">
  Nome: <input type="text" name="nome"><br>
  Email: <input type="email" name="email"><br>
  Curso:
  <select name="curso">
    <option value="ADS">Análise e Desenvolvimento de Sistemas</option>
    <option value="SI">Sistemas de Informação</option>
    <option value="BD">Banco de Dados</option>
  </select><br>
  <input type="submit" value="Enviar">
</form>
"""
```



Exemplo - formulário de inscrições

```
@app.route("/", methods=["GET"])
def index():
    return render_template_string(form_html)

@app.route("/submit", methods=["POST"])
def submit():
    nome = request.form.get("nome")
    email = request.form.get("email")
    curso = request.form.get("curso")
    print(f"Recebido: {nome}, {email}, {curso}")
    return "Formulário recebido com sucesso!", 200

if __name__ == "__main__":
    app.run(debug=True)
```

Exemplo - executando o form

► 2. Executando a aplicação localmente

No terminal:

```
bash

pip install flask
python app.py
```

Acesse no navegador:

 <http://localhost:5000>

Exemplo - código do teste de carga com Locust

```
from locust import HttpUser, task, between
import random

class FormUser(HttpUser):
    wait_time = between(1, 2)

    @task
    def preencher_formulario(self):
        self.client.get("/") # Acessa o formulário

        nome = f"Teste{random.randint(1000, 9999)}"
        email = f"usuario{random.randint(1000, 9999)}@example.com"
        curso = random.choice(["ADS", "SI", "BD"])

        payload = {
            "nome": nome,
            "email": email,
            "curso": curso
        }

        self.client.post("/submit", data=payload)
```

Exemplo - rodando o teste

Com a aplicação Flask rodando:

```
bash

pip install locust
locust -f locustfile.py --host=http://localhost:5000
```

Abra o navegador em

 <http://localhost:8089>

Preencha:

- **Number of users to simulate:** 500
- **Spawn rate:** 20

Clique em **Start swarming**.



Executando o teste

- Salve os resultados do teste junto com os códigos-fonte do formulário e do script de teste em um ZIP e envie por e-mail

Próxima aula

Automação e IA

Revisão para prova



Agradeço a sua atenção!

Qualidade de Software

Professor Prates

Aula 12

SÃO
PAULO
TECH
SCHOOL